MULTIMODAL REPRESENTATIONS

- Multimodal representations aim to learn compatible representations across different data modalities, such that semantically related inputs (e.g., an image and its description) are close in representation space
- The goal is semantic understanding and alignment, not generation
- Formally, the objective is to learn mappings

$$\boldsymbol{\varepsilon}_{\text{img}} : \boldsymbol{x}_{\text{img}} \rightarrow \mathbb{R}^d, \qquad \boldsymbol{\varepsilon}_{\text{text}} : \boldsymbol{x}_{\text{text}} \rightarrow \mathbb{R}^d$$

such that semantic similarity is preserved across modalities

WHY MULTIMODAL REPRESENTATIONS ARE NEEDED

- Text-to-image generation relies on the assumption that textual concepts and visual concepts can be meaningfully related
- However, raw pixels and raw text tokens lie in fundamentally different spaces
- Multimodal representation learning addresses this by
  - projecting both modalities into a shared embedding space
  - enforcing semantic correspondence through training objectives
- This enables cross-modal retrieval, prompt-based reasoning, and semantic guidance for generative models

JOINT EMBEDDING SPACE

In multimodal systems, each modality has its own encoder, but all encoders output vectors in the same latent space

$$\text{Image} \rightarrow z_{\text{img}} \in \mathbb{R}^d$$

$$\text{Text} \rightarrow z_{\text{text}} \in \mathbb{R}^d$$

- The learning objective ensures
  - semantically matching image–text pairs have high similarity
  - unrelated pairs have low similarity
- This creates a shared embedding space where similarity reflects meaning, not modality

MULTIMODAL ALIGNMENT

Multimodal alignment refers to the process of bringing representations from different modalities into semantic correspondence

- Alignment defines what it means for an image and text to match and determines how similarity is represented geometrically
- Well-aligned representations generalise across tasks
- Alignment is therefore a **learning problem**, not just an architectural choice

MULTIMODAL ALIGNMENT

- Multimodal alignment can be learned in two broad ways
  - **Explicit alignment:** alignment is the training objective
  - **Implicit alignment:** alignment emerges while solving another task
- Both approaches are widely used, but serve **different roles** in multimodal systems

EXPLICIT VS IMPLICIT MULTIMODAL ALIGNMENT

Aspect	Explicit Alignment	Implicit Alignment
How alignment is learned	Directly optimised during training	Emerges indirectly from another task
Training objective	Similarity between modalities	Generation or prediction loss
Supervision	Paired image–text data	Task-specific data
Representation learned	Shared embedding space	Internal feature interactions
Is similarity measurable?	Yes (e.g. cosine similarity)	No (behavioural)
Typical outputs	Image and text embeddings	Generated images or visual features
Example usage	Retrieval, ranking	Text-guided generation

Explicit alignment learns **similarity** across modalities, implicit alignment learns **influence** between modalities

MULTIMODAL REPRESENTATIONS VS CONDITIONING

Aspect	Multimodal Representation	Conditioning
Purpose	Align semantics	Guide generation
Output	Embeddings	Generated data
Usage	Before or outside generation	During generation
Core Mechanism	Contrastive learning	Attention / feature injection
Typical example	Image-text encoders	Text-to-image diffusion

Multimodal representations often **support** generative models but are not themselves generative

ALIGNMENT IN IMAGE AND TEXT ENCODERS

Multimodal alignment means that related image and text representations are close in embedding space, not identical

- Image embeddings encode **visual details** such as shape, texture, colour, and spatial layout
- Text embeddings encode **semantic abstractions** such as object identity, attributes, and relationships

Multimodal models aim for **alignment**, not **equality;** which creates unavoidable divergence

Alignment ≠ equivalence

WHY PERFECT ALIGNMENT IS IMPOSSIBLE

Modalities observe different signals. Each modality compresses information differently. Some information exists in one modality only

- For example, image shows a *dog wearing a collar* and the caption says only “a dog”
- The image contains extra information that the text does not
- Additionally, each modality introduces its own biases. Image biases include bias due to camera angle, dataset style, and lighting conditions
- Text biases include language ambiguity, word frequency effects, and descriptive omissions

These biases warp the embedding space differently



a dog

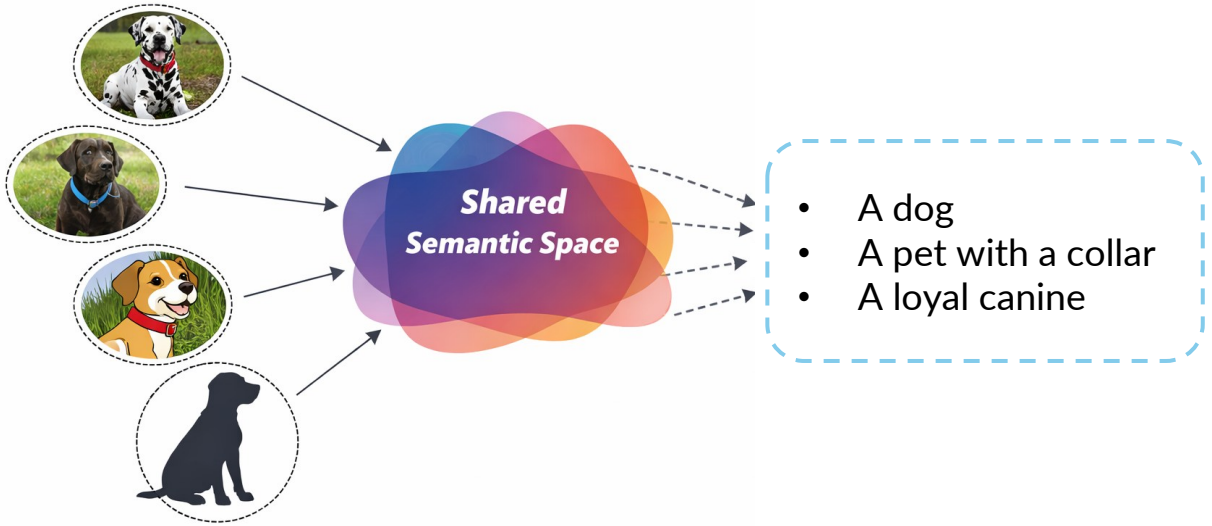
(caption should be “a dog wearing a collar”)

THE ALIGNMENT BOTTLENECK

Most multimodal models align images and text using global contrastive objectives

- Alignment is applied at the whole-image and whole-text level
- Fine-grained correspondences (e.g. *the collar*, not just *the dog*) are not explicitly aligned
- Many visually distinct images can map to the same textual concept

- This creates an alignment bottleneck, where global semantic similarity exists without precise grounding
- The model learns: "*These belong together*" **and not** *"These are the same thing"*



ALIGNMENT ISSUES

- **Retrieval Errors**
  - Text query retrieves a visually plausible but semantically wrong image
  - Image query retrieves text describing only part of the scene

PROMPT

A red traffic at a busy intersection

RETRIEVED IMAGE



Traffic signal is green

PROMPT

A dog wearing a collar

RETRIEVED IMAGE



Collar is missing

ALIGNMENT ISSUES

- **Generative Artifacts**
  - Text-to-image models prioritise language-salient features
  - Visual details not mentioned in the text may drift

PROMPT

A stack of pancakes with butter and syrup

GENERATED IMAGE



Syrup spill was not mentioned in the prompt

PROMPT

A glass of water with ice cubes and condensation on the outside

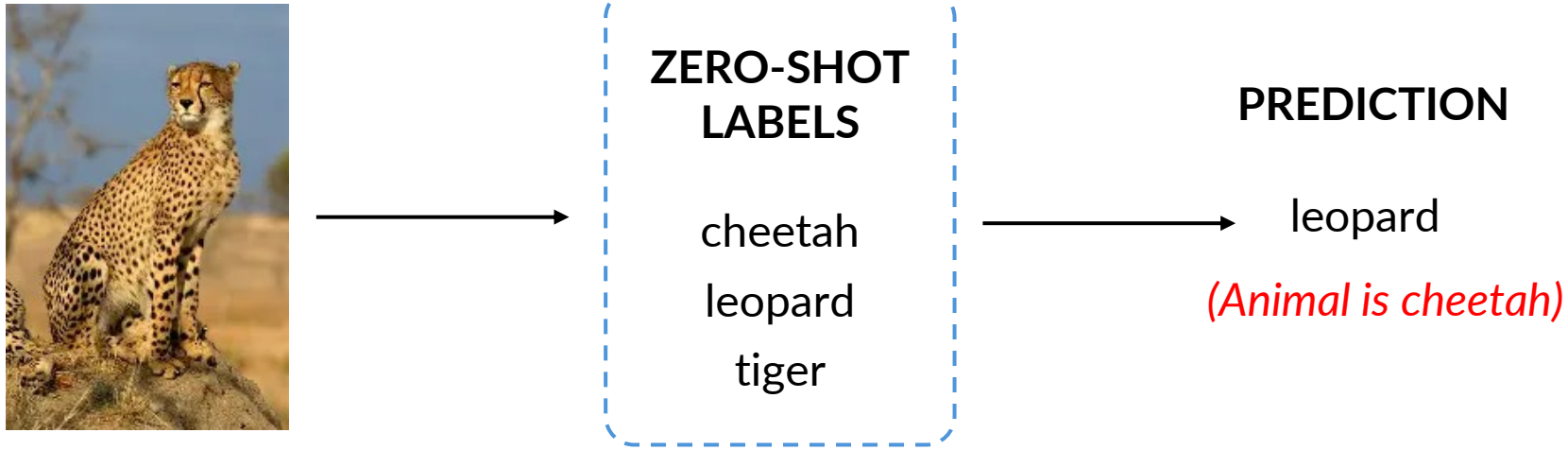
GENERATED IMAGE



- *Ice cubes and condensation on the outside is missing*
- *Water is being poured even when it is not mentioned in the prompt*

ALIGNMENT ISSUES

- **Zero-Shot Limits**
  - Works best for common, well-described concepts
  - Fails for rare, abstract, or visually subtle distinctions

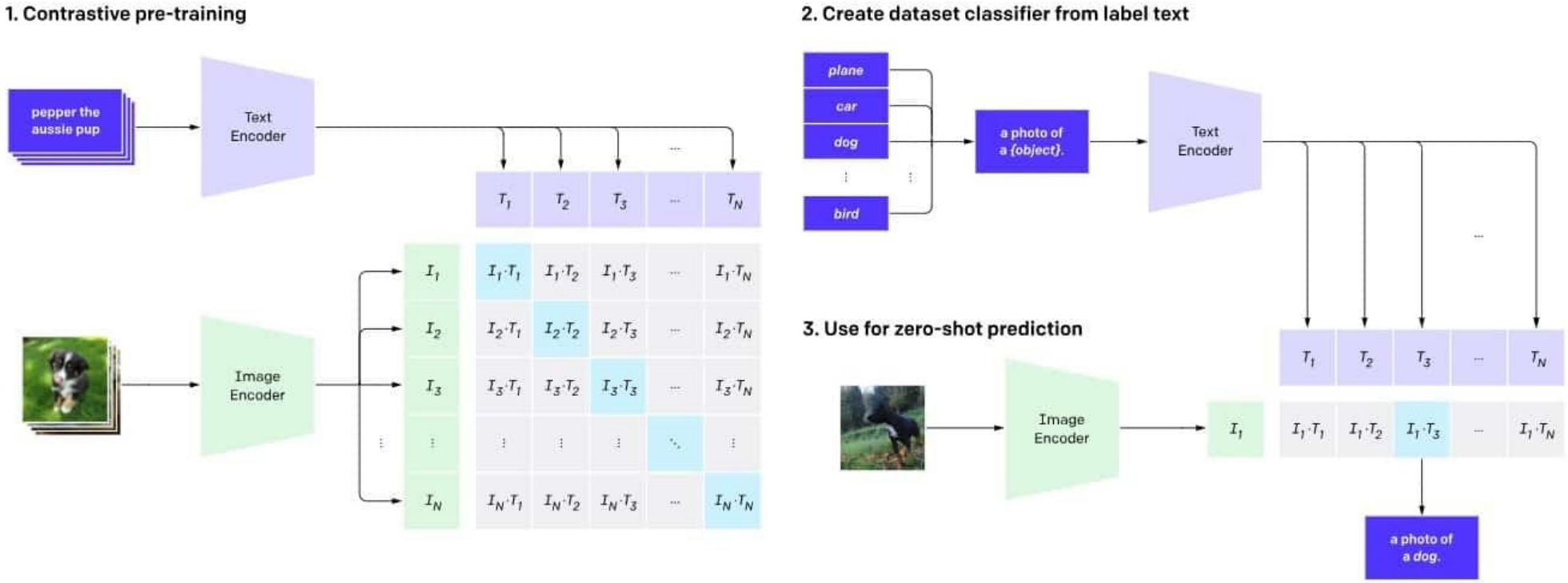


Fine-grained visual distinctions between similar categories are harder to resolve in embedding space

CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINNG (CLIP)

- CLIP (Contrastive Language–Image Pre-training) is a multimodal representation learning framework developed by OpenAI that learns to align images and text using natural language supervision
- By training on a massive dataset of image-text pairs, CLIP connects visual data with semantic meaning, unlocking capabilities similar to the zero-shot learning seen in GPT models
- CLIP is best understood as an **explicit alignment model**, not a task-specific system
- CLIP enables models to understand images through language without task-specific labels

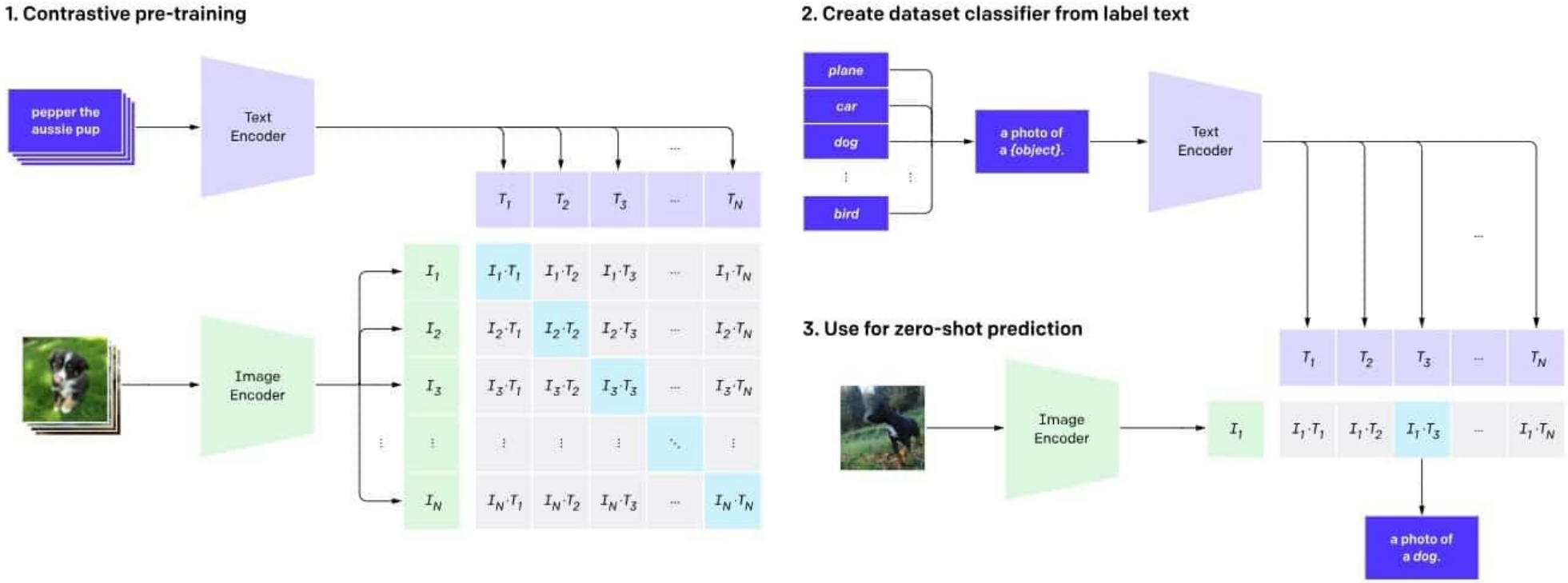
CLIP ARCHITECTURE



CLIP employs a dual-encoder framework to bridge the gap between vision and language. It utilises two distinct neural networks running in parallel

- An **Image Encoder** (typically a Vision Transformer or CNN like ResNet) that converts visual inputs into high-dimensional vectors
- A **Text Encoder** (usually a Transformer like BERT) that does the same for text captions

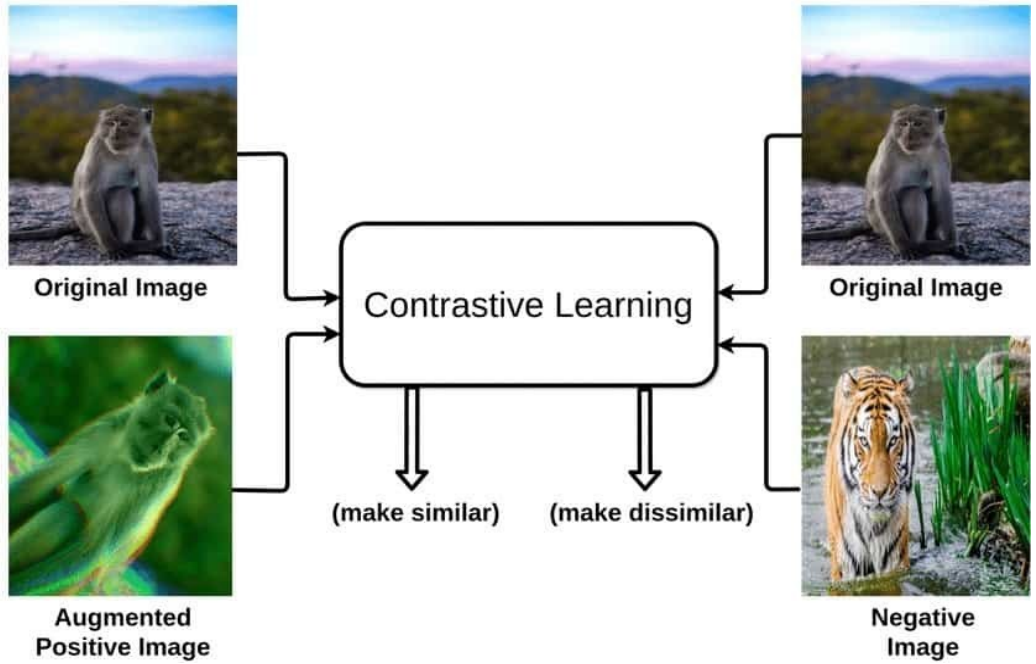
CLIP ARCHITECTURE



- The image and text encoder streams converge in a **shared embedding space**, a unified vector landscape where the model learns to align the mathematical representations of images with their corresponding textual descriptions
- This shared embedding space enables the model to understand the relationships between visual concepts and natural language
- CLIP does **not** enforce pixel-level or word-level equivalence — only semantic alignment

CLIP TRAINING PROCESS

- CLIP is trained using **contrastive learning**, where the model learns by comparing similar and dissimilar examples
- An anchor image is paired with
  - a positive match (augmented view or matching text)
  - a negative match (different image or mismatched text)
- The learning objective is to
  - make anchor–positive representations similar
  - make anchor–negative representations dissimilar
- This teaches the model semantic similarity, not class identity
- The same principle is later applied to image–text pairs in CLIP

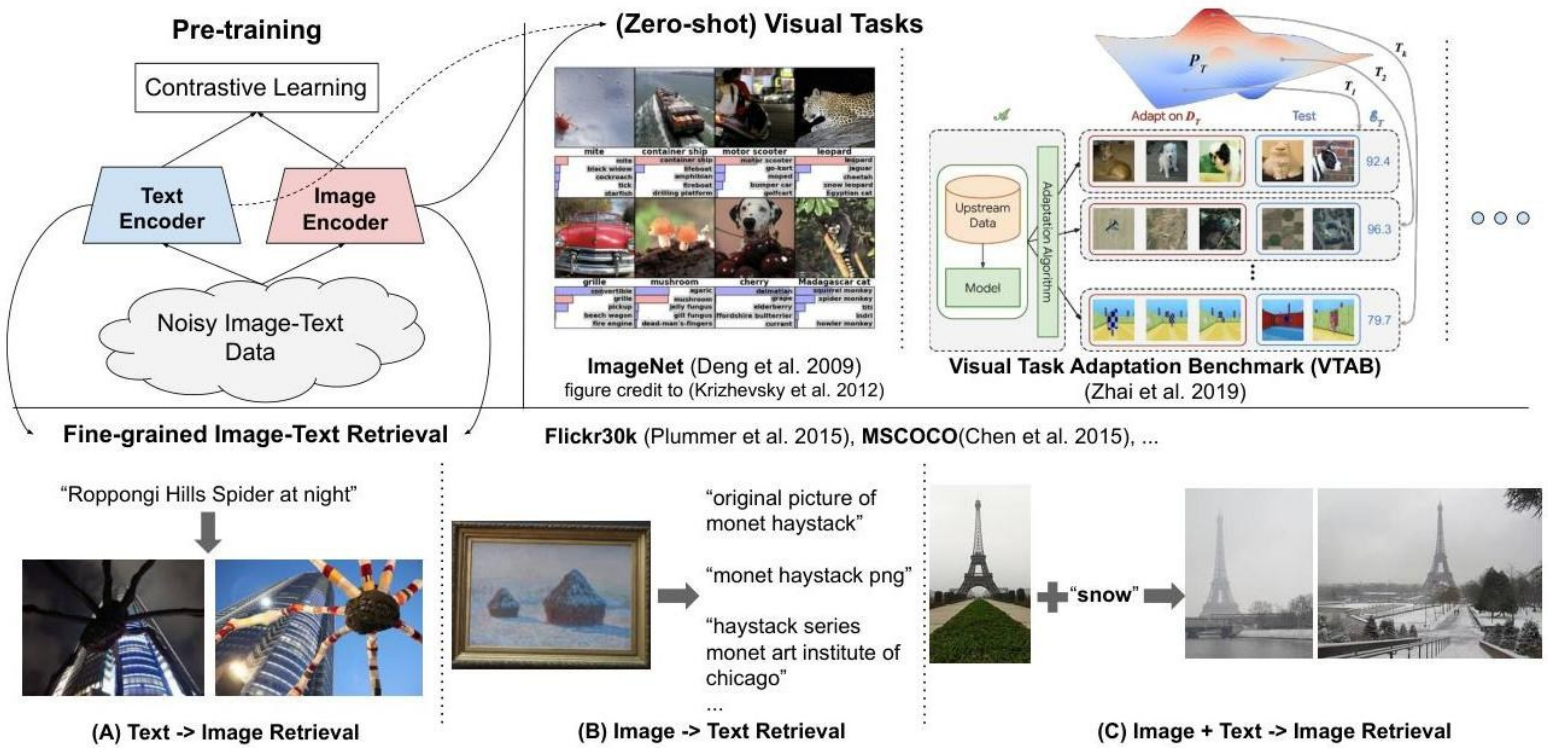


WHAT CLIP ENABLES

CLIP enables

- Zero-shot classification
- Cross-modal retrieval
- Semantic filtering and ranking
- Prompt-based scoring of generated images

These capabilities emerge without task-specific training



LIMITATIONS OF CLIP

- **Global alignment:** it aligns entire images with text descriptions and does not provide fine-grained spatial or object-level grounding
- **Limited compositional understanding:** it struggles with prompts involving complex relationships, precise counts, or attribute binding
- **Lack of reasoning capability:** it cannot perform multi-step reasoning, visual question answering, or decision-making beyond similarity comparison
- **Prompt sensitivity:** its outputs can vary significantly with changes in prompt wording, even when the semantic intent is similar
- **Non-generative nature:** it cannot generate images or language and only compares learned representations

VARIANTS OF CLIP: REPRESENTATION QUALITY AND SCALE

Variant type	What changed	What improved	Key insight
ViT-based CLIP	CNN image encoder replaced with Vision Transformer	Better global context and semantic consistency	Attention-based image encoders align well with language
Large-scale CLIP	Increased model size and training data	Stronger zero-shot generalisation	Alignment quality improves with scale
Domain-adapted CLIP	Fine-tuned on specialised datasets	Domain-specific semantic accuracy	Alignment transfers but benefits from domain grounding

These variants improve *how well concepts are represented*, while keeping the **same contrastive alignment objective**

VARIANTS OF CLIP:
BEYOND GLOBAL IMAGE-TEXT ALIGNMENT

Variant	Main idea	Limitation addressed	Typical use case
DenseCLIP	Aligns text with pixel- or patch-level features	No spatial grounding	Semantic segmentation
RegionCLIP	Aligns text with object regions	Whole-image alignment only	Open-vocabulary detection
Vision Language Detection (ViLD)	Uses CLIP as a frozen semantic classifier for regions	Fixed label sets	Open-vocabulary object detection
Frozen CLIP backbone	CLIP encoders kept fixed, task heads added	Costly retraining	Modular vision–language systems

These variants extend CLIP from **global semantic matching** to **spatial and task-aware alignment**

VISION LANGUAGE MODEL

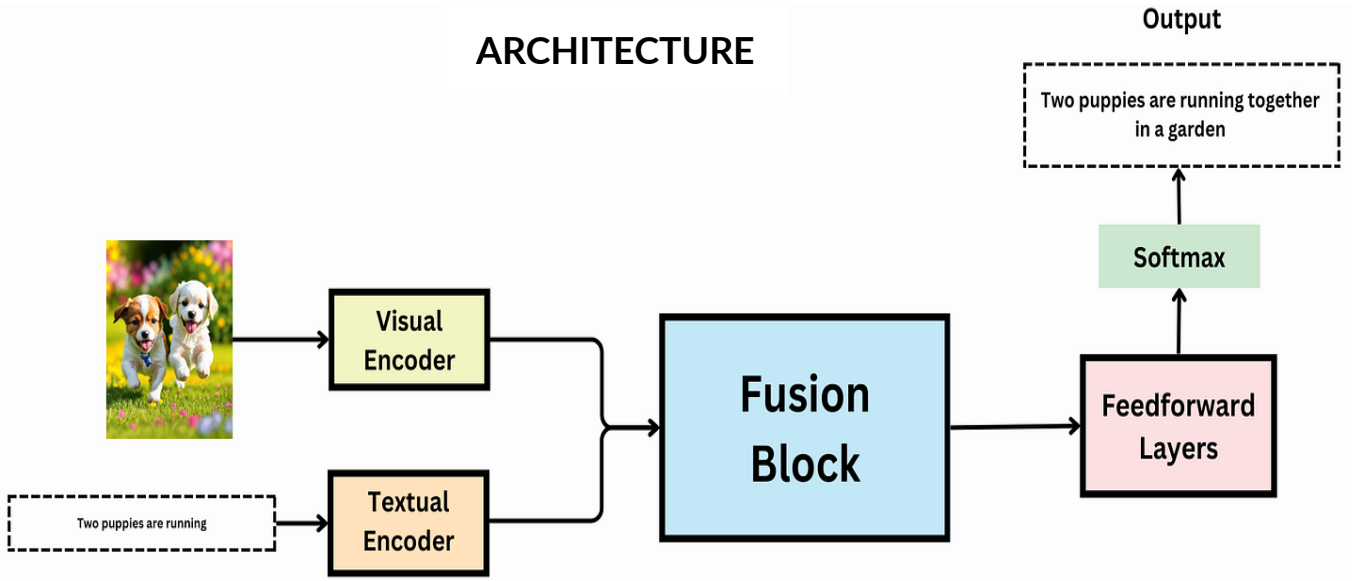
- Vision–Language Models (VLMs) represent a key step in multimodal AI, enabling models to process visual and textual data together
- They are multimodal neural architectures that jointly process image and language tokens to support grounded reasoning and generation
- By combining visual scene understanding with language-based reasoning, VLMs bridge raw imagery and human language to produce context-aware descriptions
- This goes beyond simple classification and retrieval, enabling complex reasoning for applications such as visual question answering, assistive navigation, and multimodal AI assistants

VISION LANGUAGE MODEL: COMPONENTS

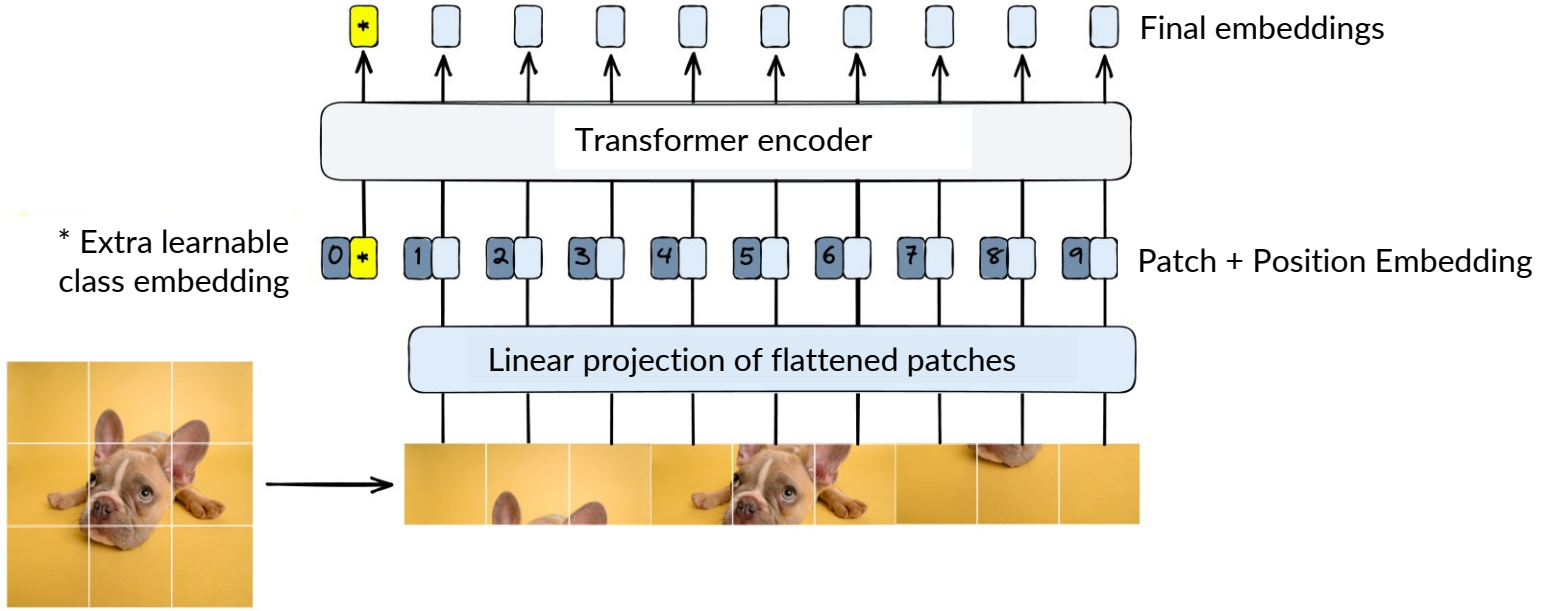
While specific architectures vary depending on the downstream task, Vision-Language Models generally rely on the following framework to synthesise multimodal data

- **Vision encoders:** converts images into visual tokens
- **Language encoders:** converts text into language tokens and performs reasoning
- **Fusion block:** it defines how and where the token sequences interact and turn into a unified, meaningful state

This is the minimal architectural scaffold needed for grounded reasoning



VISUAL ENCODERS

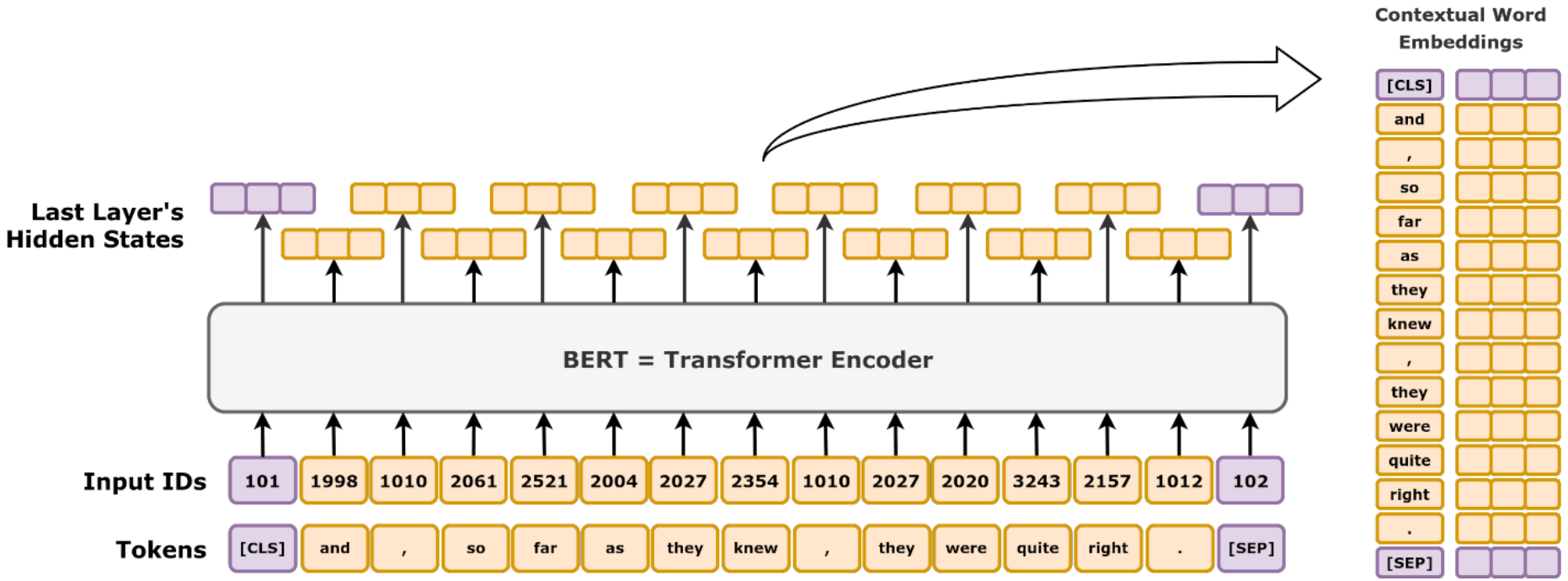


The vision encoder converts raw visual inputs such as images or videos into embeddings that can be consumed by the language model

- Early VLMs used convolutional neural networks (CNNs) for feature extraction
- Modern VLMs use Vision Transformers (ViTs) to model global context

These embeddings preserve spatial and semantic structure required for grounding

TEXT ENCODERS



Language encoder processes textual data to capture semantic meaning

- Most VLMs rely on **transformer-based LLMs** as the language backbone due to their superior ability to capture long-range dependencies and context through attention mechanisms
- In modern VLMs, the language model often serves as the **central reasoning engine**

FUSION BLOCK

Alignment alone is insufficient for tasks that require reasoning or grounding

- CLIP aligns image and text embeddings but does not support token-level interaction
- VLMs introduce **fusion layers** to allow language tokens to attend to visual tokens
- Fusion enables grounding of words in specific visual regions
- This allows question answering, spatial reasoning, and instruction following

Fusion converts aligned representations into **joint multimodal reasoning states**

FUSION MECHANISM

Fusion mechanisms describe how visual and language representations interact architecturally within a model

ADAPTER-BASED FUSION

- **Examples:** BLIP-2, LLaVA-style projector layers
- **Mechanism**
  - Vision embeddings are mapped to LLM token space using small adapter networks
  - The core LLM is mostly frozen
  - Fusion happens through learned projection + attention
- **Core idea:** Vision is aligned to language via learnable projection modules
- **When used:** Scaling VLMs without retraining the entire LLM

FUSION MECHANISM

PREFIX-BASED FUSION (a.k.a. token injection, prompt-based fusion)

- **Example:** SimVLM, PaLI-style designs
- **Mechanism**
  - Visual embeddings are projected into the same dimension as text tokens
  - They are inserted as prefix tokens to the language model
  - The transformer processes visual tokens and text tokens in one sequence
- **Core idea:** Vision becomes part of the language model's context window
- **When used:** Text generation, captioning, instruction following with images

FUSION MECHANISM

CROSS-ATTENTION FUSION

- **Example:** BLIP-2, VisualGPT, LLaVA, Flamingo (core mechanism)
- **Mechanism**
  - Language tokens query visual tokens via cross-attention layers
  - The model dynamically selects which image regions to attend to
  - Visual and textual streams interact explicitly at multiple layers
- **Core idea:** Language attends to vision during generation and reasoning
- **When used:** VQA, grounded reasoning, multimodal instruction following

FUSION MECHANISM

CO-ATTENTION / DUAL-STREAM FUSION

- **Examples:** ViLBERT, LXMERT
- **Mechanism**
  - Vision and language are encoded in separate streams
  - Cross-attention is applied in both directions
    - text attends to image
    - image attends to text
- **Core idea:** Joint reasoning through bidirectional interaction
- **When used:** Fine-grained grounding, region-level reasoning, early VLMs

FUSION MECHANISM

UNIFIED MULTIMODAL TRANSFORMER (SHARED ATTENTION FUSION)

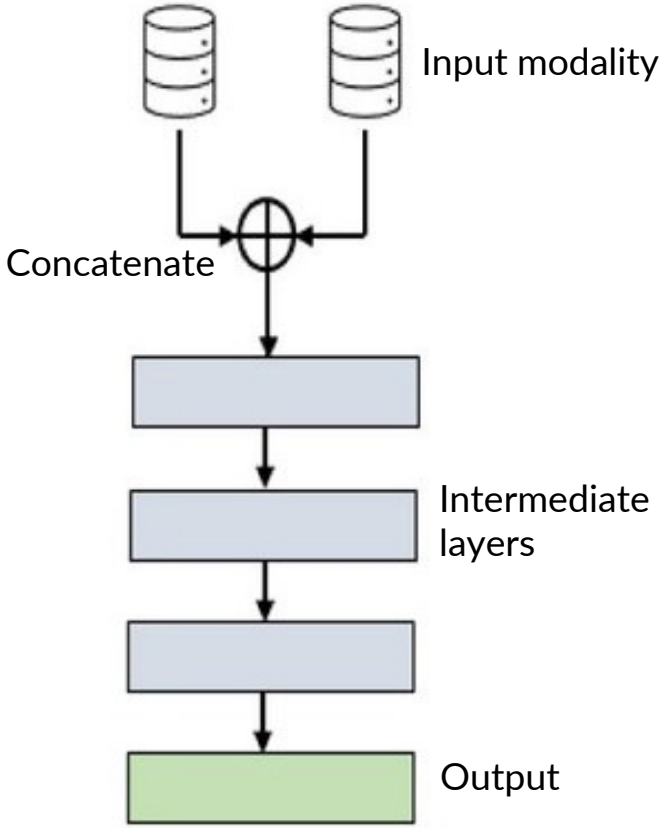
- **Examples:** GPT-4o, Gemini
- **Mechanism**
  - Visual tokens and text tokens are projected into a common embedding space
  - A single transformer processes both modalities jointly
  - Self-attention operates across all tokens without separate fusion blocks
- **Core idea:** Vision and language are modelled in a single shared representation space with joint self-attention
- **When used:** End-to-end multimodal reasoning, tightly integrated perception–language systems

FUSION DEPTH

Fusion depth describes at what stage in the network visual and language representations begin to interact

EARLY FUSION

- **Examples:** VisualBERT, UNITER
- **Mechanism**
  - Image region features and text are concatenated
  - A single transformer processes both modalities jointly from early layers
- **Core idea:** Vision and language are treated as one token sequence
- **When used:** String grounding tasks, heavy compute settings

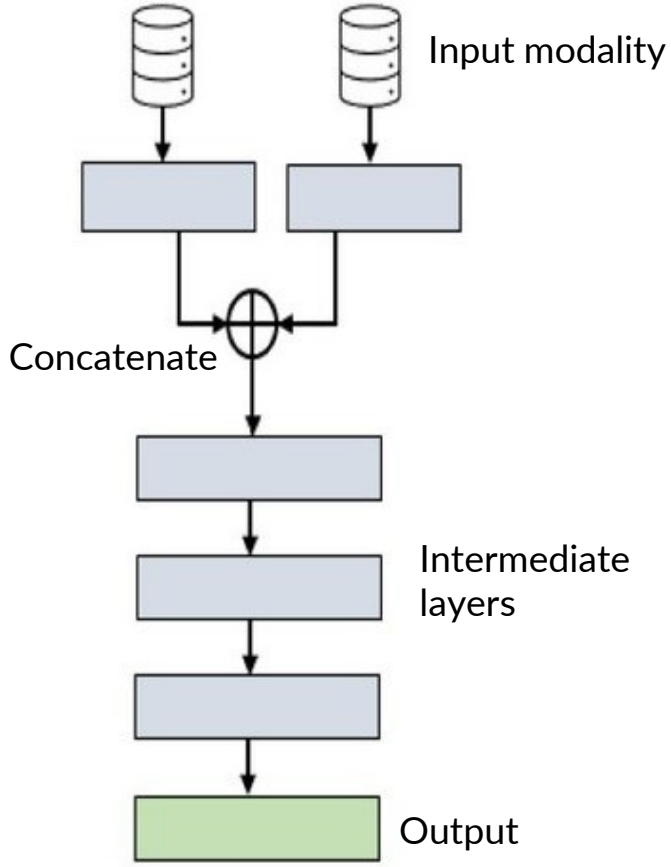


Early fusion

FUSION DEPTH

INTERMEDIATE FUSION

- **Examples:** LLaVA, BLIP-2, Flamingo
- **Mechanism**
  - Vision and language are encoded separately at lower layers
  - Fusion blocks (cross-attention) are inserted at selected depths
  - Joint representations are formed only after initial unimodal processing
- **Core idea:** Preserve modality-specific structure, then integrate for reasoning
- **When used:** Balanced grounding and efficiency, most modern VLMs

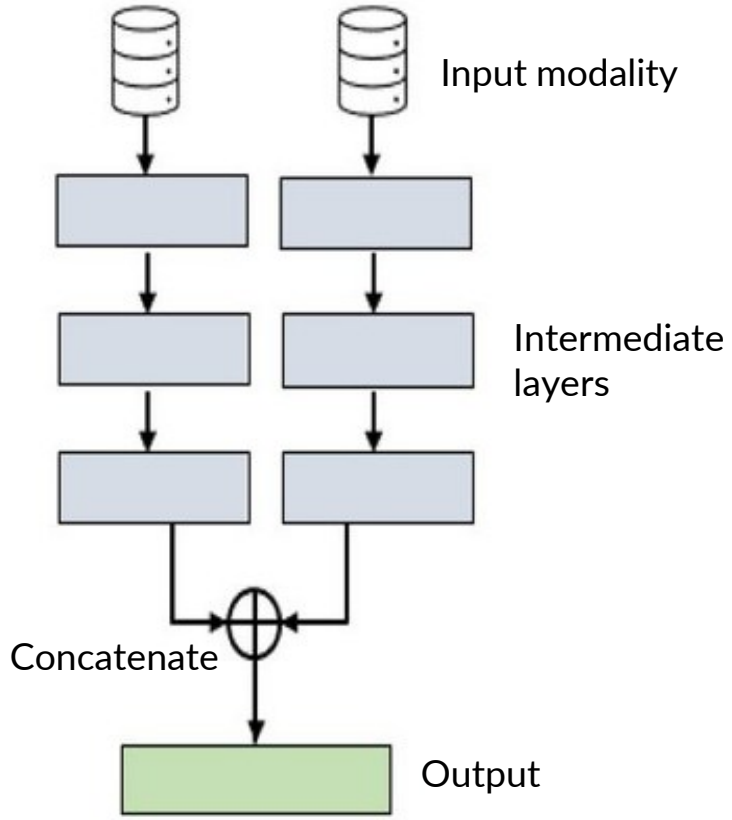


Intermediate fusion

FUSION DEPTH

LATE FUSION

- **Examples:** CLIP-based pipelines, dual-encoder + classifier systems
- **Mechanism**
  - Vision and language are processed independently
  - Interaction occurs only at similarity scoring or output layers
  - No token-level interaction during encoding
- **Core idea:** Modalities interact only at the decision stage
- **When used:** Retrieval, ranking, zero-shot classification



Late fusion

TRAINING OBJECTIVES FOR VLMs

VLMs are trained using multiple complementary objectives to align and fuse visual and language representations

- **Image–text matching (ITM):** learns to predict whether a given image–text pair is semantically aligned or mismatched
- **Contrastive alignment:** aligns global image–text representations
- **Masked modelling:** predicts masked text tokens or masked image regions
- **Generative training:** learns to generate captions, answers, or descriptions
- **Instruction tuning:** fine-tunes models to follow natural language instructions grounded in images

These objectives teach the model to use visual evidence for language generation

MASKED MODELLING AND MULTIMODAL PRETRAINING

Masked modelling enables fine-grained grounding

- **Masked language modelling:** predict missing words given image context
- **Masked image modelling:** reconstruct hidden visual patches using text context

These objectives force tight coupling between modalities

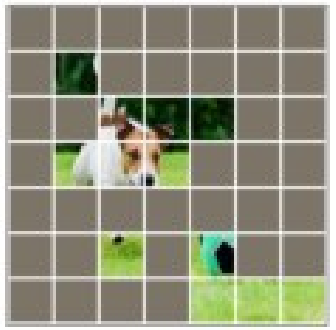
Masked language modelling



"A [MASK] is running towards a green ball"

"A dog is running towards a green ball"

Masked image modelling





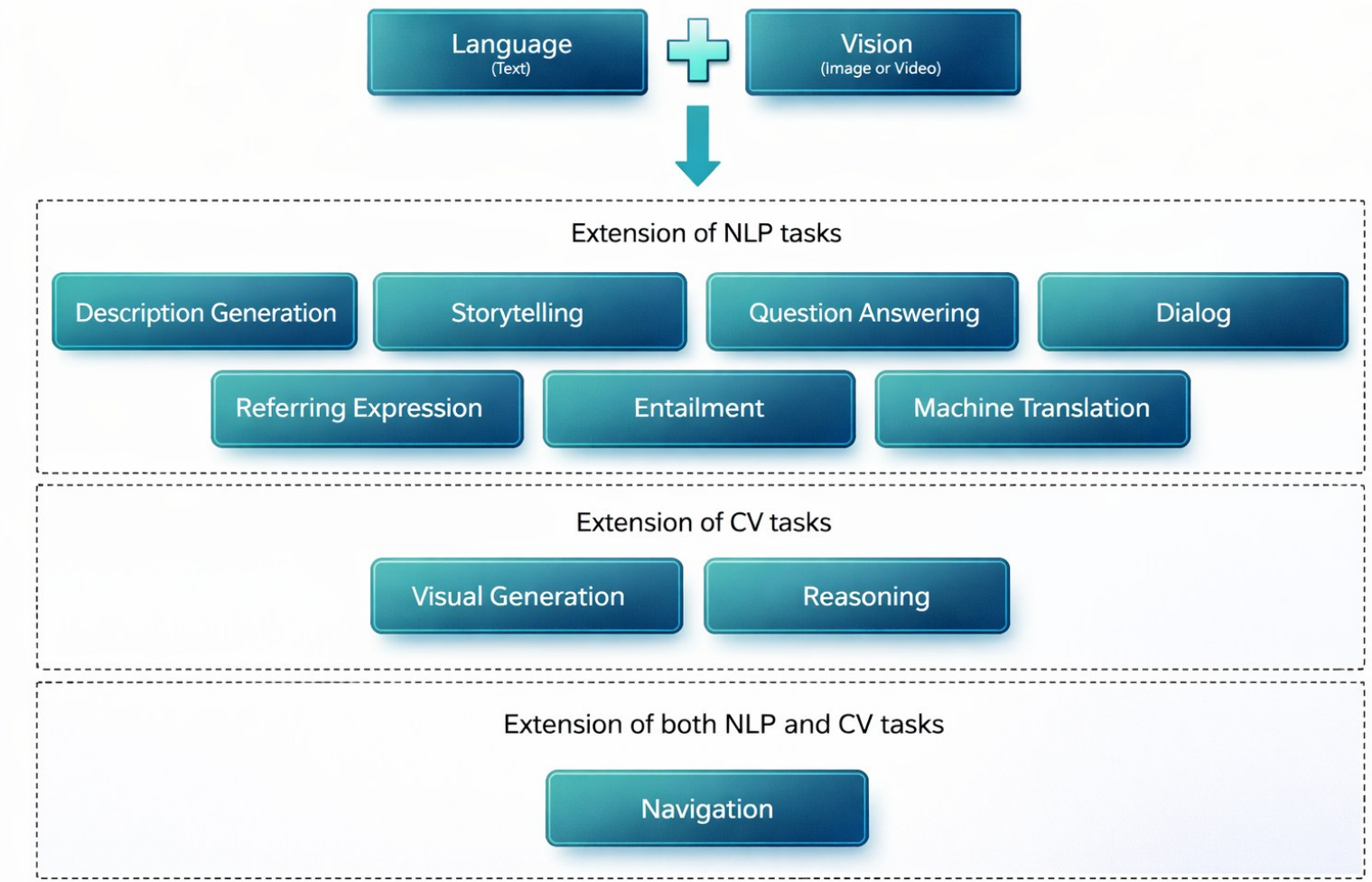
REPRESENTATIVE VLM ARCHITECTURES

Modern VLMs instantiate different architectural patterns

- **LLaVA:** ViT vision encoder + projection layer + cross-attention into LLM
- **GPT-4o:** unified multimodal model trained end-to-end
- **Gemini:** multimodal transformer supporting text, image, audio, and video
- **NVLM:** hybrid decoder-only and cross-attention architectures for efficiency

These reflect trade-offs between compute cost and reasoning capability

APPLICATIONS OF VLMs



DESIGN TRADE-OFFS AND FAILURE MODES

Architectural choices introduce trade-offs

- Strong fusion improves grounding but increases latency
- Frozen backbones improve stability but limit domain adaptation
- Unified models are flexible but expensive to train
- Spurious correlations in training data can mislead reasoning

Understanding these trade-offs is critical for system design